TECHNICAL DOCUMENTATION

CivicAI

Agentic AI-Powered Community Grievance & Resolution Platform — architecture, data flow, AI/ML components, role access, and improvement roadmap.

Version scope: canonical backend + merged role dashboards, admin capability, optional LLM tier, and proactive SLA automation

Stack: FastAPI · MongoDB (+ in-memory fallback) · React · Vite · Tailwind CSS

Prepared: September 2026

Table of Contents

1. 1 Executive Summary
2. 2 Project Structure
3. 3 Technology Stack & Libraries
4. 4 System & Data Flow
5. 5 Functional Modules (Backend)
6. 6 AI / ML Components — Deep Dive
7. 7 How Everything Is Integrated
8. 8 User Roles & Data Access
9. 9 Complaint Lifecycle (State Machine)
10. 10 Security & Privacy
11. 11 Testing & Quality
12. 12 Known Limitations
13. 13 Recommended Improvements

SECTION 1

Executive Summary

CivicAI is a full-stack civic-grievance platform: a citizen reports a problem (pothole, garbage, water leak, streetlight outage, etc.), an AI layer classifies and prioritizes it automatically, a municipal authority acts on it within tracked SLA windows, and — if the authority misses its window — an NGO partner can step in. An admin layer oversees departments, NGO organizations, user roles, and platform-wide analytics.

The system is built as one coherent application: a **React/Vite/Tailwind** frontend that talks exclusively to a **FastAPI** backend, which persists to **MongoDB** with an automatic, fully-functional in-memory fallback for offline/demo use. There is no second backend, no second database, and no second authentication system anywhere in the stack.

What's real and working today

- Full complaint lifecycle with a server-enforced state machine
- A trained text classifier + honest rule-based fallback
- An optional hosted-LLM triage tier (Gemini)
- Classical computer-vision location-match verification
- Explainable, weighted priority scoring
- Role-based dashboards for all four roles
- Proactive SLA-breach auto-escalation
- 193 automated backend tests, all passing

What's honestly a scaffold or gap

- Image-content AI ("is this actually fixed") — no trained model yet
- Department-scoped authority accounts (department is still descriptive, not access-scoping)
- Token expiration / revocation
- No automated frontend/browser test suite
- NGO identity is 1:1 with a login, not a real multi-user organization

SECTION 2

Project Structure

Backend (`backend/`)

```
backend/
├── app/
│   ├── main.py              # FastAPI app, CORS, static mount, lifespan (DB connect, SLA sweep)
│   ├── config.py             # Settings loaded from .env
│   ├── database.py           # MongoDB manager + full in-memory fallback implementation
│   ├── auth_utils.py          # Token signing/verification, require_role() dependency
│   ├── routes/                # One file per resource area — thin, delegate to services
│   │   ├── auth.py, complaints.py, authority.py, media.py,
│   │   ├── areas.py, community.py, dashboard.py,
│   │   ├── notifications.py, audit.py, admin.py
│   ├── schemas/                # Pydantic request/response shapes + enums
│   │   ├── enums.py  (ComplaintStatus, ComplaintCategory, UserRole, Department)
│   │   ├── complaint.py, user.py, area.py, media.py, admin.py, ...
│   └── services/               # ALL business logic lives here, not in routes
│       ├── ai_service.py, ml_service.py, llm_service.py
│       ├── priority_service.py, sla_service.py, sla_sweep_service.py
│       ├── geo_service.py, media_service.py, scene_match_service.py, vision_service.py
│       ├── timeline_service.py, audit_service.py, notification_service.py
│       ├── rate_limit_service.py, password_service.py, escalation_service.py
├── ml/                         # Text-classifier training pipeline (offline, run once)
│   ├── src/{prepare_dataset,train,evaluate,predict}.py
│   ├── data/, models/civicai_text_classifier/
│   └── vision/                 # Vision-model scaffold — no trained model shipped
├── tests/                      # 193 pytest tests, one file per concern
├── seed.py                     # Idempotent demo data: users, complaints, departments, orgs
└── requirements.txt
```

Frontend (`frontend/`)

```
frontend/src/
├── App.jsx                    # Route table, wraps role-gated routes in ProtectedRoute
├── routes/ProtectedRoute.jsx   # Role-based redirect (UX only — real auth is server-side)
├── context/AuthContext.jsx     # Session state, demo role switcher, login/register
├── pages/                      # One file per screen
│   ├── Home.jsx                 — public complaint feed
│   ├── ReportProblem.jsx         — citizen complaint creation + photo + AI panel
│   ├── ComplaintDetails.jsx      — full detail, timeline, confirm/reopen
│   ├── CitizenDashboard.jsx      — "my complaints", role-scoped
│   ├── AuthorityDashboard.jsx    — queue, status transitions, evidence
│   ├── NGODashboard.jsx          — escalated cases, accept, resolve
│   ├── AdminDashboard.jsx        — KPIs, departments, orgs, users, audit log
│   ├── ImpactDashboard.jsx, CommunityFeed.jsx, AreaDashboard.jsx, NearbyComplaints.jsx
│   └── Login.jsx, Register.jsx, Landing.jsx
├── components/                 # Shared UI: badges, uploader, AI panel, navbar, ...
├── services/api.js             # The ONLY place HTTP calls are made (axios wrapper)
└── utils/{constants,formatters}.js
```

SECTION 3

Technology Stack & Libraries

Backend

Library	Role
`fastapi + uvicorn`	Web framework and ASGI server; auto-generates OpenAPI docs at `/docs`
`motor + pymongo`	Async MongoDB driver — falls back to a hand-written in-memory store if unreachable
`pydantic`	Request/response validation and settings
`bcrypt`	Password hashing (with transparent migration off a legacy SHA-256 scheme)
`python-multipart`	Multipart form parsing for file uploads
`Pillow`	Image decode/verify/re-encode, EXIF stripping for public copies
`piexif`	EXIF GPS metadata extraction from uploaded photos
`opencv-python-headless`	Classical computer vision — ORB keypoint matching for scene/location verification
`scikit-learn + joblib`	Trained text classifier (TF-IDF + Logistic Regression) and model (de)serialization
`numpy`	Numerical backing for scikit-learn / OpenCV
`google-genai`	Official Gemini SDK — the optional hosted-LLM triage tier
`email-validator`	Backs Pydantic's `EmailStr` field validation
`pytest + httpx`	Test framework and the HTTP client FastAPI's `TestClient` uses internally

Frontend

Library	Role
`react + react-dom`	UI framework
`vite`	Dev server + production bundler
`react-router-dom`	Client-side routing, incl. role-gated routes
`tailwindcss + postcss + autoprefixer`	Utility-first styling — the entire visual design system
`axios`	HTTP client, one shared instance with an auth-header interceptor
`lucide-react`	Icon set used throughout every page
`clsx / tailwind-merge`	Conditional / conflict-safe className composition

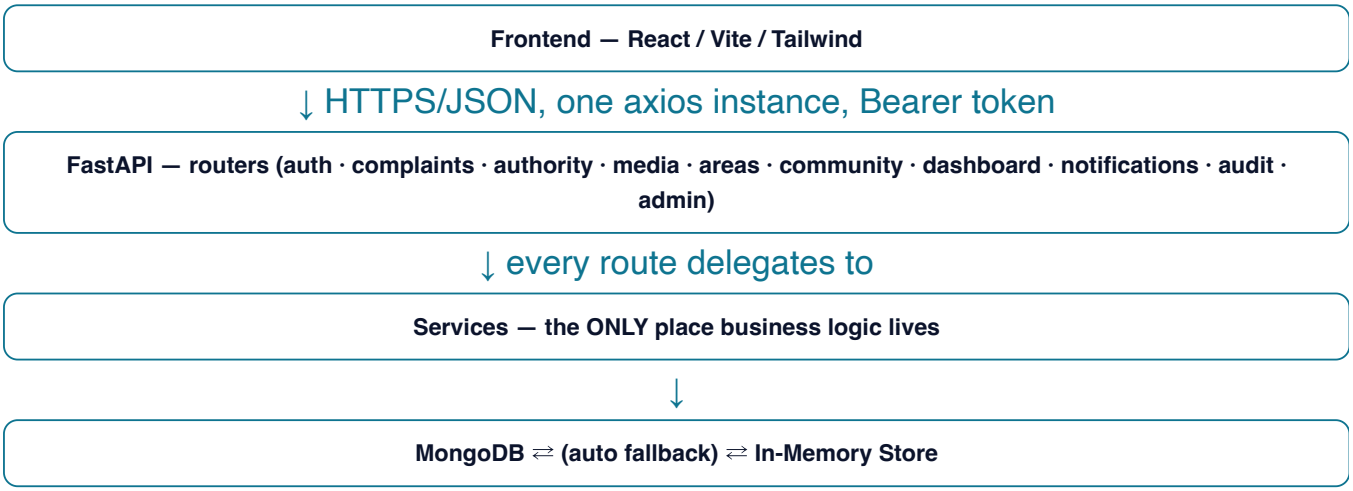
Database

MongoDB is the only application database. If it's unreachable at startup, the backend automatically switches to a **hand-written in-memory store** that implements the same async cursor API (`find` , `insert_one` , `update_one` , sorting, skip/limit, a minimal `$or` / `$in` / `$regex` matcher) — every route works identically in both modes. Collections: `users, complaints, media, supports, escalations, areas, notifications, audit_logs, timeline_events, status_history, user_area_memberships, departments, organizations` .

SECTION 4

System & Data Flow

Top-level architecture



A complaint's life, end to end

1. **Citizen submits** a report (title, description, location, optional photo). The complaint is created *hidden* from every public listing (`pendingPhoto: true`) until a photo lands.
2. **AI classification runs synchronously**, before the document is even saved — category, severity, department, and (if the optional LLM tier is on) a subcategory and human-readable summary.
3. **Priority score is computed** from severity, estimated affected users, community support, location sensitivity, and how long the issue has been open.
4. **SLA deadlines are set** (acknowledgment + resolution windows), and the complaint is **geo-resolved into an area** (polygon or nearest-circle).
5. **First photo upload** flips the complaint visible; EXIF GPS is extracted server-side, a public EXIF-stripped copy is generated, and the original (with EXIF) is stored privately.
6. **Authority acknowledges → assigns → works → resolves** (with an evidence photo and/or a note) — every transition is validated against a fixed state machine, timestamped, audited, and notifies the citizen.
7. **If the authority never acknowledges in time**, a background sweep automatically escalates the case to NGOs (or a human can escalate manually, any time).
8. **An NGO can accept** an escalated, unclaimed case (one atomic, race-safe "claim" action), then carry it through the same resolve flow.
9. **Citizen confirms or reopens** the resolution — confirming closes the case; reopening notifies every authority and puts it back in play.
10. **Every step is recorded twice**, for two different audiences: a citizen-facing *timeline* and a security-facing *audit log*.

SECTION 5

Functional Modules (Backend Routes)

Module	Purpose	Key behavior
`auth.py`	Register / login / session	Self-registration is always forced to *citizen* server-side; bcrypt hashing; rate-limited login (10/5min per email)
`complaints.py`	Core complaint CRUD + lifecycle actions	Create (runs AI+priority+SLA+geo), list (filterable incl. "my complaints"/"my claimed cases"), nearby, similar/duplicate-check, support/unsupport, evidence, verify-evidence, confirm/reopen, escalate, history, timeline
`authority.py`	Official status changes	Enforces `VALID_TRANSITIONS` state machine; also the route NGOs use to accept/resolve claimed cases; per-department/SLA-risk queue view; overview stats
`media.py`	Photo upload/list/delete	Content-validated (not extension-trust), EXIF-aware, scene-match check on evidence photos, ownership/claim-gated
`areas.py` / `community.py`	Geography & neighborhood feed	Circle and GeoJSON-polygon areas, area-scoped feed and stats, "my area" preference
`dashboard.py`	Public transparency analytics	Real aggregation only — stats, department performance, category summary, monthly trends
`notifications.py`	Per-user inbox	Unread count, mark-read, mark-all-read — ownership-checked (404, not 403, on someone else's notification)
`audit.py`	Security audit log reader	Authority/admin only
`admin.py`	Platform administration	Departments/organizations CRUD, user role changes (audited), platform overview (incl. gov-vs-NGO resolution split), system/AI-ML health

SECTION 6

AI / ML Components — Deep Dive

Every "AI decision" in CivicAI runs through `ai_service.analyze_complaint()`, a **three-tier cascade**. Each tier either produces a trustworthy answer or falls through to the next — nothing here ever blocks complaint submission, and nothing fabricates a confidence number it can't back up.

Tier 1 — Optional hosted LLM OPTIONAL, OFF BY DEFAULT

`llm_service.py` calls Google's Gemini via the official `google-genai` SDK, only if both `AI_API_KEY` and `AI_PROVIDER` are configured. It asks for a structured JSON response: category, a finer-grained subcategory, a 0–100 severity score, a priority label, a best-matching department (from the live admin-managed list), an estimated resolution time, and a one-sentence human-readable summary.

> The raw 0–100 score is **bucketed onto the same four-value `SeverityLevel` enum** every other tier uses (25/50/75 thresholds, matching the priority engine's own buckets) — a raw LLM number never leaks into a field the rest of the app treats as a fixed enum. Any failure, timeout, or out-of-enum category response causes this tier to return nothing, and the cascade falls through automatically.

Tier 2 — Trained text classifier REAL, TRAINED, WORKING

`ml_service.py` loads a model trained by `backend/ml/src/train.py`: a **TF-IDF vectorizer + Logistic Regression** "head" for each of three targets — category, severity, department — each reporting its own `predict_proba()` confidence. If the *weakest* of the three confidences is below a threshold (0.45), the result is flagged `needsReview` rather than silently trusted.

Tier 3 — Rule-based fallback classifier ALWAYS AVAILABLE

A deterministic keyword-matching classifier (`ai_service.fallback_analyze_text`) that never fails and never needs a model file. It matches complaint text against category-specific keyword sets (pothole/road → Road Infrastructure, garbage/waste → Waste Management, etc.), and boosts severity when the text mentions a sensitive location (school, hospital, market).

Priority Engine EXPLAINABLE, WEIGHTED SCORING

Not a model — a transparent formula, recalculated on every relevant change:

```
Priority = 0.40 × Severity
         + 0.20 × Affected Users
         + 0.15 × Community Support
         + 0.15 × Location Sensitivity
         + 0.10 × Duration Open
```

Every score ships with its own breakdown, a dominant-factor explanation ("driven mainly by its assessed severity"), and a calculation version — so a priority number is always accountable to a visible reason, not a black box.

Computer Vision — Scene/Location Match REAL, CLASSICAL CV, WORKING

`scene_match_service.py` uses OpenCV's **ORB keypoint detector + RANSAC homography** to answer one narrow, real question: does an authority's "after" evidence photo show the *same physical location* as the citizen's original "before" photo? This runs automatically on every evidence upload — free, local, no API key, no model download.

Computer Vision — Image-Content Understanding HONEST SCAFFOLD — NOT TRAINED

`vision_service.py` is real, working infrastructure for a genuinely different, harder question: "does this photo show the issue actually fixed?" It checks for `torch` / `transformers` and a trained model file, and today **always and correctly reports itself unavailable** — no paid API and no labelled before/after dataset exist yet to train one. This is reported honestly as `available: false`, never faked.

Duplicate / Similar-Complaint Detection REAL, WORKING

Combines geo-proximity (primary signal, when location is known) with **TF-IDF cosine text similarity** (secondary signal / sole signal without location) to warn a citizen "someone nearby already reported this," with named reasons per match rather than a bare score.

Training pipeline (offline, not run per-request)

```
backend/ml/
├── src/prepare_dataset.py   # builds train/test split from labels.json
├── src/train.py              # fits the 3 TF-IDF+LogisticRegression heads, saves .joblib +
metadata.json
├── src/evaluate.py           # honest accuracy/confidence reporting — documents dataset limits, not
just numbers
├── src/predict.py            # CLI smoke-test of a trained model
└── vision/                   # parallel scaffold for a future image-content model — no data yet
```

SECTION 7

How Everything Is Integrated

Complaint **creation** and complaint **lifecycle transitions** are the two places every cross-cutting service meets:

On creation

1. AI cascade (§6) → category/severity/department
2. Priority engine → weighted score
3. SLA service → ack/resolution deadlines
4. Geo service → area resolution, GeoJSON point
5. Timeline + audit → two independent event records

On every status change / action

1. Server-side state-machine validation
2. Timeline event (citizen-facing) — role-aware public/internal split
3. Audit log entry (security-facing) — sanitized, never leaks secrets
4. Notification — to the affected citizen, or a role-wide broadcast
5. Rate limiting on sensitive actions (login, uploads, admin writes)

Three independent observer services fan out from *every* meaningful action, and none can block it or replace another:

Service	Audience	Purpose
`timeline_service`	Citizen + privileged roles	"What happened to my complaint" — role-aware (citizens see public messages only; authority/NGO/admin also see internal metadata)
`audit_service`	Authority/Admin, security review	"Who did what, when" — a closed action vocabulary, forbidden-key sanitized (passwords/tokens never logged)
`notification_service`	The relevant user(s)	Per-recipient inbox rows; a role-wide broadcast fans out to every current user of that role

The **proactive SLA sweep** (a background `asyncio` task, not a separate process) and the **manual escalate button** both call the exact same `escalation_service.perform_escalation()` — one implementation, two triggers, so there is never a risk of the two drifting apart.

SECTION 8

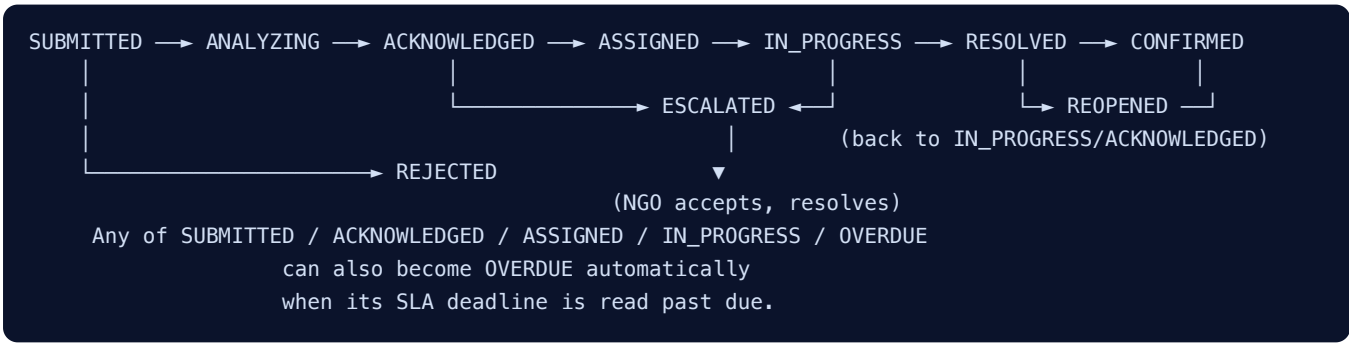
User Roles & Data Access

Role	Dashboard	Can do	Cannot do
Citizen	`/` (public feed), `/my-complaints`	Report issues, upload photos, support others' complaints, view public feed/nearby/community, confirm or reopen *their own* resolved complaints, escalate their own stuck complaint	Change any official status, see private originals of others' photos, act on any complaint they didn't file
Authority (Government)	`/authority`	Acknowledge/assign/progress/resolve any complaint, upload evidence, view the full authority queue & overview stats	Manage users/departments/organizations (admin-only)
NGO	`/ngo`	Accept an unclaimed *escalated* case (one atomic action), then act on *that case only* exactly like an authority would (evidence, resolve)	Touch any complaint it hasn't claimed; act on a case another NGO already claimed
Admin	`/admin`	Create departments, register NGO organizations, change any user's role, view platform KPIs (incl. gov-vs-NGO resolution split), view system/AI health, view the security audit log	Everything is audit-logged; no route bypasses server-side RBAC

Frontend role-gating is UX, not security. A logged-in citizen who edits the URL to `/authority` gets redirected client-side — but the real boundary is server-side: every sensitive route re-checks the caller's signed token and role independently. No permission is ever granted by the frontend alone.

SECTION 9

Complaint Lifecycle (State Machine)



Every transition is checked against a fixed table server-side — an invalid jump (e.g. straight from `SUBMITTED` to `RESOLVED`) is rejected with a clear error, never silently accepted. `RESOLVED` additionally requires either a note or an existing evidence photo — an authority (or an NGO on its claimed case) cannot close a case with nothing to show for it.

SECTION 10

Security & Privacy

- **Passwords:** bcrypt, with transparent on-login migration off an older unsalted SHA-256 scheme.
- **Sessions:** a compact signed token (`user_id.role.hmac_sha256`) verified with constant-time comparison — no JWT library, no external session store, and role is *never* taken from client input.
- **Authorization:** a `require_role(...)` dependency on every sensitive route, plus explicit ownership/claim checks in the route body where "whose data is this" matters.
- **Media privacy:** every uploaded photo is stored twice — a private original (EXIF/GPS intact) reachable only by its owner or an authority, and a public, EXIF-stripped, downsized copy for everyone else.
- **Rate limiting:** login attempts, uploads, and admin writes are all throttled per-user/per-key.
- **Audit trail:** a dedicated security log, separate from the citizen-facing timeline, with forbidden fields (passwords, tokens) stripped at write time.

SECTION 11

Testing & Quality

193 automated backend tests, all passing, run against an isolated in-memory database that never touches a real cluster. Coverage spans authentication, the full complaint lifecycle, every role's permission boundary, media validation, evidence verification, geolocation, community features, dashboards, the admin API, the optional LLM tier (including its failure/fallback paths), and the proactive SLA sweep (including a direct test of its race-safety at the database layer).

> There is currently **no automated frontend/browser test suite**. Every UI page's correctness today is verified by code review and a successful production build, not a rendered screenshot — a real, acknowledged gap, listed again in the next section.

SECTION 12

Known Limitations

Area	Limitation
Authority accounts	Not department-scoped — every authority sees every complaint; `assignedTo` is a free-text field, not a real linked account
Sessions	Signed tokens never expire and can't be revoked
NGO identity	An NGO is one login account, not a multi-user organization; `claimedByOrgId` exists on the schema but is never populated yet
Vision AI	No trained image-content model exists; only the honest scaffold and the classical-CV location-match check are real
Frontend testing	No automated UI/E2E test suite
Scale	Several aggregate/overview endpoints fetch up to 10,000 documents in memory rather than using a true database-side aggregation pipeline
Search	Regex-based OR-query across fields, not a real text index — fine at today's scale, won't scale indefinitely
LLM tier	Implemented and unit/integration-tested against mocks and the real SDK's call shape, but never exercised against the live Gemini API (no key provisioned in the build/test environment)

SECTION 13

Recommended Improvements

AI / ML — highest leverage

- **Collect a real, labelled complaint dataset.** The trained text classifier's severity head currently scores near-chance on its synthetic training data — this is the single highest-leverage AI improvement available, ahead of any model-architecture change.
- **Upgrade duplicate detection past TF-IDF** to sentence embeddings, so paraphrased reports ("deep hole in the road" vs "pothole") are actually recognized as duplicates, not just exact-keyword overlaps.
- **Train a real image-content model** (or wire a vision-capable LLM call) once a labelled before/after photo dataset exists, to finally give `/verify-evidence` a real "is this fixed" signal instead of only a location-match check.
- **Calibrate the LLM tier's confidence** instead of the current fixed placeholder value, and add response-schema validation tests against the live Gemini API once a key is available.
- **Feed real admin-managed departments into every tier**, not just the optional LLM one, so a newly created department is discoverable by the trained/fallback classifiers too.

Security / Architecture

- **Token expiration and refresh.** Today's signed tokens never expire — a real deployment needs a session-length decision and revocation path.
- **Department-scoped authority accounts**, so a water-department officer only sees water complaints, not the entire municipal queue.
- **Move rate limiting to a shared store** (e.g. Redis) before horizontally scaling past one backend process — today's limiter is correctly documented as process-local.
- **Real database-side aggregation** for the dashboard/overview endpoints, removing the 10,000-document in-memory ceiling.
- **A genuine NGO-organization membership model** (multiple staff per org, org-level case assignment) to replace today's one-login-equals-one-NGO simplification.

Testing / Ops

- **Stand up a frontend test runner** (Vitest + React Testing Library, or a Playwright E2E suite) — the single biggest testing gap relative to the backend's 193-test coverage.
- **Add real-Gemini-key smoke tests** to CI (behind a secret, skipped when absent) so the LLM tier's live response shape is checked automatically, not only its mocked integration logic.
- **Rotate the demo** `SECRET_KEY` and any credentials before any deployment beyond local development.

Product / UX

- **A real citizen profile page** — neither the original project nor the merged capabilities currently have one.
- **Push/real-time notifications** (WebSocket or SSE) instead of poll-on-load, especially valuable for authority/NGO queues under load.
- **Department/organization editing and deactivation** in the admin UI (today only creation exists, matching what was explicitly asked for — edit/delete is a natural next increment).

This document reflects the codebase as of the current merged state (canonical FastAPI/MongoDB backend plus the ported Citizen/NGO/Admin dashboards, optional LLM tier, and proactive SLA sweep). Generated as a technical reference — not a public-facing or marketing document.